1. Objetivo y recomendaciones.

Este ejercicio contribuye al aprendizaje de la programación concurrente de procesos en Java.

2. Indicaciones de entrega.

Esta tarea es individual.

Realiza todos los ejercicios, agrúpalos en un paquete y entrégalos en el Aula Virtual.

El nombre del archivo debe incluir tu nombre y apellido, sin acentos ni caracteres especiales. Por ejemplo, el nombre del archivo podría ser:
SanchezAndres_PSP_Tarea_1_3

3. Tarea: Programación de las clases Process y ProcessBuilder de Java

Realiza los siguientes ejercicios de programación concurrente en Java y elabora tus apuntes de estudio:

Actividad A: Prueba el programa de ejemplo con los siguientes casos:

a) El proceso se ejecuta y no produce salida, debido a errores del argumento de entrada. Por ejemplo, con el argumento `ls /etc` desde Ubutu [`bash -c "ls"`] o con *`dir` desde `cmd` en Windows* [`cmd /c dir`]

b) El argumento/programa que se pasa, no existe y el proceso no se ejecuta. Por ejemplo, con el argumento `programa_que_no_existe`

c) El proceso que se ejecuta, produce un error. Por ejemplo, con el argumento `ls /et` desde Ubutu o con `di` desde `cmd` en Windows.

Programa de ejemplo:

```java
package lanzaproceso;

import java.io.IOException;
import java.util.Arrays;

public class LanzaProceso {
    Run | Debug
    public static void main(String[] args){
        // hay que comprobar que entra como argumento algún comando del SO
        if (args.length <= 0) {
            System.out.println(x:"Debes pasar un comando a ejecutar");
            System.exit(status:1);
        }
        ProcessBuilder pb = new ProcessBuilder(args);
        pb.inheritIO();
        /*
         * inheritIO()
         * Hace que el proceso herede la entrada y salida estándares de su proceso padre
         */
        try {
            Process p = pb.start();
            int codRet = p.waitFor();
            /*
             *  waitFor()
             *  Espera a que temine la ejecución del proceso hijo y
             *  devuelve un código
             */
            System.out.println("La ejecución de " + Arrays.toString(args)
                + " devuelve " + codRet
                + " " + (codRet==0 ? "(ejecución correcta)" : "ERROR")
                );
        }
        catch (IOException e) {
            System.err.println(x:"Error durante la ejecución del proceso");
            System.err.println(x:"Información detallada");
            System.err.println(x:"----------------------");
            e.printStackTrace();
            System.err.println(x:"----------------------");
            System.exit(status:2);
        }catch (InterruptedException e) {
            /*
             * Si se interrumpe la ejecución del programa, se lanza una excepción
             * del tipo InterruptedException que se capturaría aquí!!
             */
            System.err.println(x:"Proceso interrumpido");
            System.exit(status:3);
        }
    }
}//Actividad 1.3
```

Actividad B: Crea un programa que lance un proceso que tenga un bucle para tardar en terminar y a la vez, compruebe si se sigue ejecutando. Debes tener en cuenta lo siguiente:

- El programa debe comprobar cada 3 segundos(nota 1), si el proceso que ha lanzado sigue en ejecución.
- El programa debe terminar cuando detecte, usando `isAlive()`, que el proceso ya no esté en ejecución
- Al terminar, el programa debe presentar un mensaje de salida.
- Debes probarlo en Ubuntu y en Windows.

(nota 1) Para hacer una pausa, esperando un determinado tiempo, se puede usar `Thread.sleep(int tiempo_ms)`

Actividad C: Crea un programa para Windows y Ubuntu en java que muestre cuál es el directorio por defecto de ejecución de un proceso. Debes usar el método `directory()` de `ProcessBuilder`.

El programa también debe ejecutar un mismo comando *(ej: `ls` en Ubuntu o `dir` en Windows)* en distintos directorios, de tu elección *(ej: directorio actual y directorio c:)*, esta vez con el método `directory(File directorio)` de `ProcessBuilder`.

Actividad D: Crea una aplicación en java que muestre el entorno de ejecución de un proceso, Se debe usar `Map<String, String> environement()` de `ProcessBuilder`. Hay que iterar sobre el `Map<String, String>` devuelto para mostrar cada entrada.

Programa de ejemplo:

```java
        // El escape en java para caracteres especiales es \

        /** en Ubuntu:
         * ProcessBuilder pb = new ProcessBuilder(
         *      new String[] {"find",  "/", "-name", "\"*\""});
         */

        // en Windows:
        ProcessBuilder pb = new ProcessBuilder(
                new String[] {"cmd", "\\", "*"});
        System.out.printf(format:"Se ejecuta el comando: %s\n", Arrays.toString(args));

        /*
         * Con .inheritIO()
         * el proceso hijo hereda la entrada y salida del padre
         */
        pb.inheritIO();

        /*
         * Con .redirectErrorStream(true)
         * la salida de error se redirige a la salida estandar.
         * Si no se hace esto, el mensaje de que se ha terminado la ejecución
         * del programa pasa desapercibido, porque sale entremezclado con la salida
         * de error, que muestra cantidad de mensajes de error
         */
        pb.redirectErrorStream(redirectErrorStream:true);
        try {
            Process p = pb.start();

            if (!p.waitFor(MAX_TIEMPO, TimeUnit.MILLISECONDS)){
                p.destroy();
                System.out.printf(format:"\n\n!!!AVISO: No ha terminado en %d msg\n", MAX_TIEMPO);
            }
        } catch (IOException e) {
            System.err.println(x:"Error durante la ejecución. Información detallada");
            System.err.println(x:"-----------------------------------------------");
            e.printStackTrace();
            System.err.println(x:"-----------------------------------------------");
            System.exit(status:1);
        }catch (InterruptedException ex) {
            System.err.println(x:"Proceso Interrumpido");
            System.exit(status:2);
            }
    }
}
```

4. Criterios de puntuación. Total 10 puntos.

- Actividad A [2 puntos]
- Actividad B [2 puntos]
- Actividad C [2 puntos]
- Actividad D [2 puntos]
- Limpieza y bien comentado el código [2 puntos]